BÀI TOÁN K-ARMED BANDIT TRONG REINFORCEMENT LEARNING

1 Giới thiệu bài toán k-armed bandit

Đặc điểm quan trọng nhất phân biệt *Reinforcement Learning* (Học tăng cường) với các loại hình học máy khác là nó sử dụng thông tin huấn luyện để *đánh giá* (evaluate) các hành động đã thực hiện, thay vì *hướng dẫn* (instruct) hành động nào là đúng. Phản hồi đánh giá (evaluative feedback) chỉ cho biết hành động được thực hiện tốt đến mức nào, chứ không cho biết đó có phải là hành động tốt nhất hay tồi tệ nhất có thể hay không. Điều này tạo ra nhu cầu cần phải chủ động *khám phá* (active exploration) để tìm kiếm các hành vi tốt.

Bài toán **k-armed bandit** (trong tài liệu gốc sử dụng ký hiệu *n-armed bandit*) là một phiên bản đơn giản hóa, mang tính chất phi liên kết (nonassociative) của Reinforcement Learning. Đây là bài toán cơ sở giúp chúng ta hiểu rõ nhất cách thức hoạt động của phản hồi đánh giá.

Định nghĩa bài toán: Hãy tưởng tượng bạn phải đối mặt liên tục với việc lựa chọn một trong k tùy chọn (hoặc hành động) khác nhau. Sau mỗi lần lựa chọn, bạn nhận được một phần thưởng (reward) bằng số, được rút ra từ một phân phối xác suất tĩnh (stationary probability distribution) phụ thuộc vào hành động bạn vừa chọn. Mục tiêu của bạn là tối đa hóa tổng phần thưởng kỳ vọng (expected total reward) trong một khoảng thời gian dài, ví dụ như qua 1000 lần chọn hành động (hay còn gọi là *time steps*).

Nguồn gốc tên gọi: Tên gọi *k-armed bandit* bắt nguồn từ sự tương đồng với máy đánh bạc (slot machine) hay còn gọi là "máy cướp một tay" (one-armed bandit). Tuy nhiên, bài toán của chúng ta có k cần gạt thay vì chỉ một. Mỗi lần chọn hành động tương đương với việc kéo một cần gạt, và phần thưởng là số tiền trúng giải. Bằng cách thử lặp đi lặp lại, mục tiêu là tối đa hóa tiền thắng cược bằng cách tập trung kéo những cần gạt mang lại lợi ích cao nhất.

2 Các khái niệm, định nghĩa và ký hiệu trong thiết lập bài toán

Để giải quyết bài toán toán học này, chúng ta cần định nghĩa các ký hiệu chuẩn xác như sau:

- t: Bước thời gian (time step), $t = 1, 2, 3, \ldots$
- A_t: Hành động (Action) được lựa chọn tại bước thời gian t.

- R_t: Phần thưởng (Reward) nhận được tại bước thời gian t, sau khi thực hiện hành động A_t.

Trong bài toán k-armed bandit, mỗi hành động a có một phần thưởng trung bình hoặc kỳ vọng (expected or mean reward) khi hành động đó được chọn. Ta gọi đây là **giá trị (value)** của hành động đó, ký hiệu là $q(a)$.

Về mặt toán học, $q(a)$ chính là **giá trị kỳ vọng (Expected Value)** của phần thưởng R_t khi biết hành động được chọn là a. Giá trị kỳ vọng trong thống kê là giá trị trung bình theo trọng số xác suất của tất cả các kết quả có thể xảy ra. Công thức được viết là:

$$q(a) \doteq \mathbb{E}[R_t \mid A_t = a] \tag{1}$$

Nếu bạn biết chính xác giá trị $q(a)$ của từng hành động, bài toán sẽ trở nên vô cùng tầm thường: bạn chỉ việc luôn chọn hành động có giá trị cao nhất. Tuy nhiên, trong thực tế, chúng ta không biết chắc chắn các giá trị này. Ta chỉ có các **giá trị ước lượng (estimated value)** của hành động a tại bước thời gian t, được ký hiệu là $Q_t(a)$. Mục tiêu là làm cho $Q_t(a)$ càng gần với $q(a)$ càng tốt.

Dựa trên các ước lượng này, tại bất kỳ bước thời gian nào, luôn có ít nhất một hành động có giá trị ước lượng lớn nhất. Đây được gọi là hành động **greedy** (tham lam).

- **Khai thác (Exploitation)**: Khi bạn chọn hành động greedy, bạn đang khai thác kiến thức hiện tại của mình để tối đa hóa phần thưởng tức thời.
- **Khám phá (Exploration)**: Khi bạn chọn một trong các hành động phi tham lam (nongreedy), bạn đang khám phá để cải thiện ước lượng về giá trị của các hành động đó trong tương lai.

Sự xung đột (conflict) giữa Exploration và Exploitation là một thách thức cốt lõi đặc trưng của Reinforcement Learning. Khai thác mang lại phần thưởng ngắn hạn, nhưng Khám phá lại mang về phần thưởng tổng thể lớn hơn trong dài hạn.

3 Phương pháp Action-value, Greedy và ϵ-greedy

3.1 Phương pháp Action-value và Trung bình mẫu

Các phương pháp **Action-value** là các phương pháp ước lượng giá trị của hành động và sử dụng các ước lượng đó để đưa ra quyết định lựa chọn hành động.

Cách tự nhiên nhất để ước lượng giá trị thật $q(a)$ là tính trung bình các phần thưởng thực tế nhận được mỗi khi hành động a được chọn. Cụ thể, nếu tại bước thời gian t, hành động a đã được chọn $N_t(a)$ lần, tạo ra các phần thưởng $R_1, R_2, \ldots, R_{N_t(a)}$,

thì giá trị ước lượng của nó là:

$$Q_t(a) \doteq \frac{R_1 + R_2 + \cdots + R_{N_t(a)}}{N_t(a)} \tag{2}$$

Nếu $N_t(a) = 0$, ta có thể gán cho $Q_t(a)$ một giá trị mặc định (ví dụ: $Q_1(a) = 0$). Theo *Luật số lớn (Law of large numbers)* trong xác suất thống kê, khi số lần chọn $N_t(a) \to \infty$, giá trị trung bình mẫu $Q_t(a)$ sẽ hội tụ về giá trị kỳ vọng thực tế $q(a)$. Phương pháp này được gọi là phương pháp **trung bình mẫu (sample-average)**.

3.2 Phương pháp Greedy

Quy tắc lựa chọn hành động đơn giản nhất là luôn chọn hành động có ước lượng Action-value cao nhất tại thời điểm hiện tại. Nếu có nhiều hành động có cùng giá trị lớn nhất, ta có thể chọn ngẫu nhiên một trong số chúng. Cú pháp toán học của hành động tham lam A_t^* được viết là:

$$A_t \doteq \arg\max_a Q_t(a) \tag{3}$$

Trong đó, toán tử $\arg\max_a$ biểu thị giá trị của biến a làm cho biểu thức theo sau đạt cực đại.

Phương pháp **greedy** luôn luôn *khai thác* (exploit) kiến thức hiện tại để tối đa hóa phần thưởng tức thì; nó không dành thời gian để thử nghiệm các hành động trông có vẻ kém hơn để xem liệu chúng có thực sự tốt hơn hay không.

3.3 Phương pháp ϵ-greedy

Một giải pháp thay thế đơn giản là hành xử greedy hầu hết mọi lúc, nhưng thi thoảng, với một xác suất nhỏ ϵ (epsilon), ta thay vì chọn greedy lại chọn ngẫu nhiên một cách độc lập từ tất cả các hành động với xác suất bằng nhau (bao gồm cả việc có thể vô tình chọn lại hành động greedy). Phương pháp sử dụng quy tắc chọn lựa gần như tham lam này được gọi là **ϵ-greedy**.

Lợi thế của phương pháp ϵ-greedy là khi số bước tiến tới vô cùng, mọi hành động sẽ được thử vô số lần (nghĩa là $N_t(a) \to \infty$ cho mọi a), đảm bảo rằng tất cả các $Q_t(a)$ sẽ hội tụ chính xác về $q(a)$.

3.4 Phân tích ví dụ 10-armed Testbed

Để so sánh hiệu quả của greedy và ϵ-greedy, tài liệu đề xuất bài thử nghiệm **10-armed testbed**. Trong bài thử nghiệm này, có 2000 bài toán bandit được tạo ngẫu nhiên với $k = 10$ hành động.

- **Phân phối xác suất**: Đối với mỗi bài toán, giá trị thực $q(a)$ được lấy mẫu từ một phân phối chuẩn (Gaussian distribution) với giá trị trung bình là 0 và phương sai là 1, ký hiệu là $\mathcal{N}(0, 1)$.

- Phần thưởng R_t tại bước t cũng được cộng thêm một nhiễu ngẫu nhiên phân phối chuẩn với trung bình là 0 và phương sai là 1, tức là $R_t \sim \mathcal{N}(q(A_t), 1)$.

Kết quả:

- **Phương pháp Greedy ($\epsilon = 0$):** Tăng trưởng phần thưởng rất nhanh ở những bước đầu, nhưng sau đó đi ngang và dừng lại ở mức thấp. Lý do là phương pháp greedy thường xuyên bị kẹt ở các hành động tối ưu cục bộ (suboptimal actions). Nó chỉ tìm được hành động tối ưu trong khoảng 30% số lần chạy.

- **Phương pháp ϵ-greedy với $\epsilon = 0.1$:** Khám phá nhiều hơn nên nhanh chóng tìm ra hành động tối ưu. Tuy nhiên, vì nó luôn chọn ngẫu nhiên 10% số lần, hiệu suất bị giới hạn không vượt quá 91% lựa chọn tối ưu.

- **Phương pháp ϵ-greedy với $\epsilon = 0.01$:** Cải thiện chậm hơn so với $\epsilon = 0.1$ ở giai đoạn đầu do tần suất khám phá ít hơn. Tuy nhiên, về dài hạn, phương pháp này sẽ vượt qua $\epsilon = 0.1$ và đạt đỉnh hiệu năng cao hơn vì xác suất chọn sai chỉ còn 1%.

3.5 Triển khai tăng dần (Incremental Implementation)

Để tính toán $Q_t(a)$ theo phương pháp trung bình mẫu (Công thức 2), nếu giữ nguyên toàn bộ lịch sử phần thưởng, yêu cầu bộ nhớ và tính toán sẽ tăng tiến tính theo thời gian. Ta có thể giải quyết việc này bằng công thức **cập nhật tăng dần** (chỉ cần tính toán $O(1)$ về bộ nhớ).

Giả sử ta chỉ xét một hành động duy nhất. Gọi Q_n là giá trị ước lượng sau $n-1$ lần hành động đó được chọn (nghĩa là trung bình của $n-1$ phần thưởng đầu tiên), và R_n là phần thưởng nhận được ở lần chọn thứ n. Trung bình của n phần thưởng được tính như sau:

$$\begin{aligned} Q_{n+1} &= \frac{1}{n}\sum_{i=1}^{n} R_i \\ &= \frac{1}{n}\left(R_n + \sum_{i=1}^{n-1} R_i\right) \\ &= \frac{1}{n}\left(R_n + (n-1)Q_n + Q_n - Q_n\right) \\ &= \frac{1}{n}\left(R_n + nQ_n - Q_n\right) \\ &= Q_n + \frac{1}{n}\Big[R_n - Q_n\Big] \end{aligned} \tag{4}$$

(Lưu ý: Trong tài liệu gốc ký hiệu là k thay vì n cho số thứ tự lần thử. Ở đây dùng n để tránh nhầm lẫn với k trong k-armed bandit).

Quy tắc cập nhật này có dạng tổng quát vô cùng quan trọng trong suốt lĩnh vực Reinforcement Learning:

$$NewEstimate \leftarrow OldEstimate + StepSize\Big[Target - OldEstimate\Big] \tag{5}$$

Trong đó biểu thức $[Target - OldEstimate]$ chính là **sai số (error)** của ước lượng. Ước lượng cũ sẽ dịch chuyển một bước (*StepSize*, ở đây là $\frac{1}{n}$) hướng về phía giá trị mục tiêu (*Target*, ở đây là R_n).

4 Exercise 2.1

Đề bài: Trong sự so sánh được thể hiện ở Hình 2.1 (Figure 2.1), phương pháp nào sẽ hoạt động tốt nhất trong dài hạn (in the long run) xét về phần thưởng tích lũy (cumulative reward) và xác suất tích lũy của việc chọn hành động tốt nhất (cumulative probability of selecting the best action)? Nó sẽ tốt hơn bao nhiêu? Hãy biểu diễn câu trả lời của bạn một cách định lượng (quantitatively).

Lời giải chi tiết:

Trong dài hạn (khi $t \to \infty$), phương pháp **ϵ-greedy với $\epsilon = 0.01$** sẽ là phương pháp hoạt động tốt nhất.

Để giải thích định lượng cho điều này, chúng ta cần tính xác suất để mỗi phương pháp chọn được hành động tối ưu (optimal action) sau khi nó đã có đủ thời gian để khám phá và tìm ra chính xác đâu là hành động tốt nhất.

Trong thuật toán ϵ-greedy, ở bất kỳ bước thời gian nào, tác tử (agent) sẽ:

- Chọn hành động *greedy* (hành động tốt nhất theo đánh giá hiện tại) với xác suất là $1 - \epsilon$.
- Khám phá (*explore*) bằng cách chọn ngẫu nhiên một trong tất cả k hành động (bao gồm cả việc có thể vô tình chọn lại hành động tốt nhất) với xác suất ϵ. Do đó, xác suất vô tình chọn trúng hành động tốt nhất trong lúc khám phá là $\frac{\epsilon}{k}$.

Bài toán *10-armed testbed* có $k = 10$. Xác suất tiệm cận (asymptotic probability) để chọn hành động tốt nhất trong dài hạn, ký hiệu là $P(\text{optimal})$, được tính bằng công thức:

$$P(\text{optimal}) = (1 - \epsilon) + \frac{\epsilon}{k} \tag{6}$$

1. Phân tích về xác suất chọn hành động tốt nhất:

- **Với phương pháp $\epsilon = 0.1$:**

$$P_{0.1}(\text{optimal}) = (1 - 0.1) + \frac{0.1}{10} = 0.9 + 0.01 = 0.91 \text{ (hay 91\%)} \tag{7}$$

- **Với phương pháp $\epsilon = 0.01$:**

$$P_{0.01}(\text{optimal}) = (1 - 0.01) + \frac{0.01}{10} = 0.99 + 0.001 = 0.991 \text{ (hay 99.1\%)} \quad (8)$$

Kết luận về xác suất: Trong dài hạn, phương pháp $\epsilon = 0.01$ sẽ chọn hành động tốt nhất với tần suất cao hơn phương pháp $\epsilon = 0.1$ là:

$$0.991 - 0.91 = 0.081 \text{ (tức là cao hơn chính xác 8.1\%)} \quad (9)$$

2. Cumulative Reward:

Bởi vì phương pháp $\epsilon = 0.01$ chọn hành động tối ưu thường xuyên hơn **8.1%** so với phương pháp $\epsilon = 0.1$, phần thưởng trung bình mỗi bước (average reward per step) của nó trong dài hạn cũng sẽ cao hơn tương ứng.

Cụ thể, gọi q^* là giá trị kỳ vọng của hành động tốt nhất. Với sự chênh lệch 8.1% trong xác suất chọn q^*, tốc độ gia tăng của phần thưởng tích lũy (độ dốc của đường cumulative reward) của phương pháp $\epsilon = 0.01$ sẽ dốc hơn. Theo thời gian $t \to \infty$, khoảng cách về tổng phần thưởng tích lũy giữa $\epsilon = 0.01$ và $\epsilon = 0.1$ sẽ ngày càng nới rộng ra vô hạn (tốt hơn $8.1\% \times q^*$ cho mỗi bước thời gian).

*(Lưu ý: Phương pháp *greedy* $\epsilon = 0$ là tệ nhất trong dài hạn vì nó bị mắc kẹt ở các hành động *suboptimal* trong khoảng 2/3 số trường hợp và ngừng hoàn toàn việc cải thiện).*

5 Thiết lập đánh giá "10-armed Testbed"

Để so sánh hiệu quả của các phương pháp *Greedy* và *ϵ-greedy*, giáo trình của Sutton & Barto sử dụng một bộ khung đánh giá tiêu chuẩn gọi là **10-armed Testbed**. Đây là một môi trường mô phỏng giúp chúng ta quan sát hành vi của thuật toán trong điều kiện có nhiễu.

5.1 Cấu trúc của bộ thử nghiệm

Thiết lập này bao gồm 2000 bài toán k-armed bandit độc lập với $k = 10$. Đối với mỗi bài toán:

- **Giá trị thực ($q_*(a)$):** Đối với mỗi hành động $a \in \{1, \ldots, 10\}$, giá trị thực được chọn ngẫu nhiên từ một **Phân phối chuẩn (Gaussian Distribution)** với trung bình bằng 0 và phương sai bằng 1:

$$q_*(a) \sim \mathcal{N}(0, 1) \quad (10)$$

- **Phần thưởng (R_t):** Khi tác tử chọn hành động A_t tại bước thời gian t, phần thưởng thực tế R_t sẽ được lấy mẫu từ một phân phối chuẩn có trung bình là

$q_*(A_t)$ và phương sai là 1:

$$R_t \sim \mathcal{N}(q_*(A_t), 1) \tag{11}$$

- **Quy mô:** Mỗi phương pháp được chạy trong 1000 bước thời gian (time steps) và được lặp lại qua 2000 phiên bản bài toán khác nhau để lấy giá trị trung bình.

5.2 Tại sao sử dụng thiết lập này để đánh giá?

Thiết lập 10-armed Testbed không phải là một bài toán ngẫu nhiên đơn thuần, nó mang những ý nghĩa thống kê quan trọng để đánh giá học máy:

1. **Tính khách quan thông qua Trung bình mẫu (Statistical Significance):** Nếu chỉ chạy một bài toán duy nhất, kết quả có thể do may mắn (ví dụ: tác tử chọn trúng máy tốt nhất ngay từ đầu). Bằng cách lấy trung bình trên 2000 bài toán, các yếu tố nhiễu ngẫu nhiên bị triệt tiêu, giúp ta thấy được hiệu suất thực sự của thuật toán.

2. **Đánh giá khả năng Exploration vs Exploitation:** Vì phần thưởng R_t có độ lệch chuẩn là 1, nên phần thưởng nhận được từ hành động tốt nhất và hành động kém hơn có thể bị chồng lấn lên nhau. Điều này bắt buộc thuật toán phải có cơ chế **khám phá** đủ mạnh để không bị đánh lừa bởi những phần thưởng cao ngẫu nhiên từ các máy tồi.

3. **Khả năng hội tụ:** Thiết lập này cho phép quan sát tốc độ hội tụ. Một phương pháp tốt không chỉ cần tìm ra máy tốt nhất mà còn phải tìm ra nó thật nhanh để tối đa hóa phần thưởng tích lũy.

4. **Đo lường xác suất hành động tối ưu (Optimal Action Probability):** Vì chúng ta là người thiết lập môi trường, chúng ta biết chính xác máy nào là tốt nhất (q_* nào lớn nhất). Từ đó, ta có thể vẽ biểu đồ tỉ lệ phần trăm số lần tác tử chọn đúng máy tốt nhất theo thời gian — đây là thước đo chuẩn xác nhất cho sự thông minh của thuật toán.

5.3 Phân tích kết quả thực nghiệm

Dựa trên kết quả từ thiết lập này (Hình 2.1 trong tài liệu):

- Phương pháp **Greedy** nhanh chóng bị bão hòa vì nó thường xuyên bị kẹt vào một hành động có vẻ tốt ở giai đoạn đầu nhưng thực tế lại không phải là tối ưu.

- Phương pháp ϵ**-greedy** tiếp tục cải thiện vì nó không bao giờ ngừng khám phá. Ngay cả khi ϵ nhỏ, nó vẫn đảm bảo rằng về mặt lý thuyết, xác suất tìm thấy máy tối ưu sẽ tiến dần đến 100% khi $t \to \infty$.

Ký hiệu và thuật ngữ

Dưới đây là bảng tổng hợp các ký hiệu toán học và thuật ngữ chuyên ngành được sử dụng trong bài viết này:

Ký hiệu	**Ý nghĩa**
k	Số lượng hành động (số "tay" của máy đánh bạc).
t	Bước thời gian (Time step).
A_t	Hành động được lựa chọn tại thời điểm t.
R_t	Phần thưởng nhận được sau khi thực hiện hành động A_t.
$q(a)$	Giá trị thực (giá trị kỳ vọng) của hành động a.
$Q_t(a)$	Giá trị ước lượng của hành động a tại thời điểm t.
$N_t(a)$	Số lần hành động a đã được chọn tính đến trước thời điểm t.
$\mathbb{E}[X]$	Giá trị kỳ vọng của biến ngẫu nhiên X.
$\doteq$	Ký hiệu định nghĩa (Bằng theo định nghĩa).
ϵ	Tham số xác suất dành cho việc khám phá (Exploration).
$\arg\max_a$	Giá trị của a làm cho biểu thức đi kèm đạt cực đại.
$\mathcal{N}(\mu, \sigma^2)$	Phân phối chuẩn (Gaussian) với trung bình μ và phương sai σ^2.
α (hoặc $1/n$)	Tham số kích thước bước (Step-size) trong cập nhật tăng dần.
Exploitation	Khai thác: Chọn hành động tốt nhất hiện tại.
Exploration	Khám phá: Thử các hành động mới để thu thập thông tin.
Stationary	Tĩnh: Phân phối phần thưởng không thay đổi theo thời gian.

Ghi chú: Các ký hiệu được tuân thủ theo tiêu chuẩn của giáo trình Reinforcement Learning: An Introduction (Sutton & Barto).